

0.70.1. Plus fluide : la lettre au pixel près — de base, ailleurs

Le 0.70 avec le B, parti de (20, 20).

1. Ce qu'il faut comprendre

C'est le **0.70**, avec le **B**, parti de **(20, 20)** pixels, en haut à gauche.

C'est la **version de base** : la deuxième place, seule. Le 0.70.2 met les deux ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **340 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          55 o ← la console, à l'allumage - la mise en route de la console
main()             113 o ← démarrage - ta fonction
VBlank              8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage       28 o ← image() - attend l'image suivante
AttendreVBlank      16 o ← démarrage - attend que l'écran soit libre
RangerLutins        11 o ← démarrage - range les 40 lutins hors de l'écran
InstallerTransfert  15 o ← démarrage - installe la copie des lutins en mémoire rapide
CopierTuiles        31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
LireManette         37 o ← bouton() - lit les huit boutons
Tuiles              16 o ← CopierTuiles - 1 lettre sur 43
DonneesTransfert    10 o ← InstallerTransfert - la petite routine qui copie les lutins
```

Lettres gravées : B

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```
sprite() × 1      23 o sur place, dans main()
bouton() × 4      20 o sur place, dans main() + LireManette (37 o)
image() × 1       6 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <bouton>      // lit un bouton de la manette
#include <sprite>      // place un lutin de 8 × 8 au pixel près
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — PX

```
uint8_t px = 20;    // le B, en pixels
```

3. UNE VARIABLE — PY

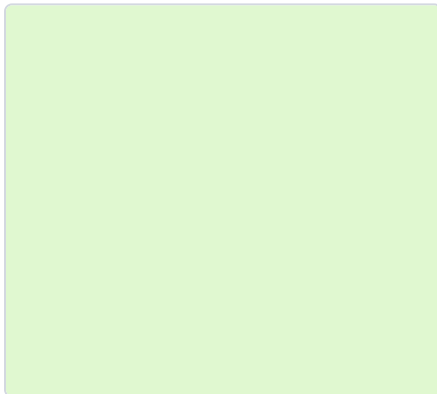
```
uint8_t py = 20;
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

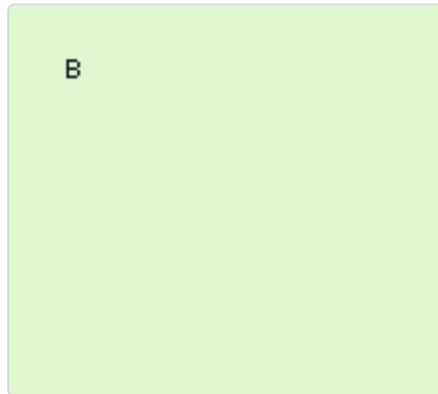
```
int main() {
    while (true) {
        image();
        if (bouton(DROITE) && px < 152) px++;
        if (bouton(GAUCHE) && px > 0)   px--;
        if (bouton(BAS) && py < 136)   py++;
        if (bouton(HAUT) && py > 0)    py--;
        sprite(0, px, py, ALPHABET[1]); // le lutin n° 0
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



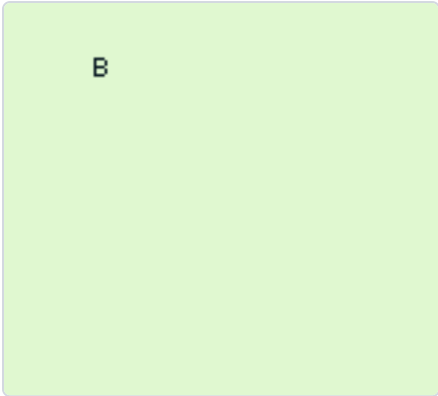
la toute première image — la console vient de démarrer — image 1



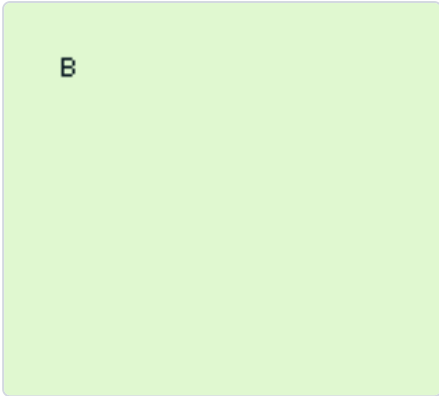
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

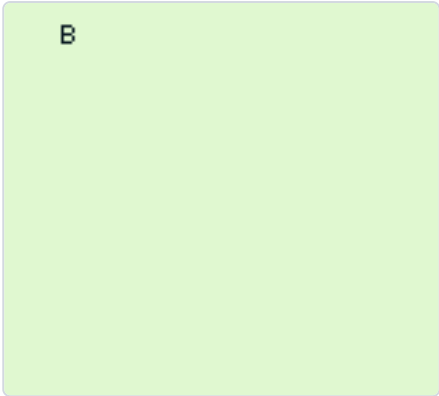
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

A green square with a black letter 'B' in the top-left corner.

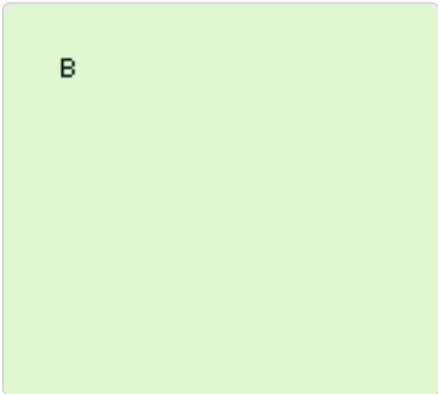
DROITE — l'écran a changé

A green square with a black letter 'B' in the top-left corner.

GAUCHE — l'écran a changé

A green square with a black letter 'B' in the top-left corner.

HAUT — l'écran a changé

A green square with a black letter 'B' in the top-left corner.

BAS — l'écran a changé

5. Ce qu'on doit voir

Un B en haut à gauche, qui glisse avec les flèches.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le lutin 0 a glissé de 30 pixels

Ensuite : **0.70.2. Plus fluide : la lettre au pixel près — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++